Smart pointers – auto_ptr until C++17 !

Smart Pointers (Modern C++)

In modern C++ programming, the Standard Library includes smart pointers, which are used to help ensure that programs are free of memory and resource leaks and are exception-safe.

Uses for smart pointers

Smart pointers are defined in the `std` namespace in the <memory> header file. They are crucial to the RAII or *Resource Acquisition Is Initialization* programming idiom. The main goal of this idiom is to ensure that resource acquisition occurs at the same time that the object is initialized, so that all resources for the object are created and made ready in one line of code. In practical terms, the main principle of RAII is to give ownership of any heap-allocated resource—for example, dynamically-allocated memory or system object handles—to a stack-allocated object whose destructor contains the code to delete or free the resource and also any associated cleanup code.

In most cases, when you initialize a raw pointer or resource handle to point to an actual resource, pass the pointer to a smart pointer immediately. In modern C++, raw pointers are only used in small code blocks of limited scope, loops, or helper functions where performance is critical and there is no chance of confusion about ownership.

The following example compares a raw pointer declaration to a smart pointer declaration.

C++
```
void UseRawPointer()
{
    // Using a raw pointer -- not recommended.
    Song* pSong = new Song(L"Nothing on You", L"Bruno Mars");

    // Use pSong...

    // Don't forget to delete!
    delete pSong;
}


void UseSmartPointer()
{
    // Declare a smart pointer on stack and pass it the raw pointer.
    unique_ptr<Song> song2(new Song(L"Nothing on You", L"Bruno Mars"));
```

```
    // Use song2...
    wstring s = song2->duration_;
    //...

} // song2 is deleted automatically here.
```

As shown in the example, a smart pointer is a class template that you declare on the stack, and initialize by using a raw pointer that points to a heap-allocated object. After the smart pointer is initialized, it owns the raw pointer. This means that the smart pointer is responsible for deleting the memory that the raw pointer specifies. The smart pointer destructor contains the call to delete, and because the smart pointer is declared on the stack, its destructor is invoked when the smart pointer goes out of scope, even if an exception is thrown somewhere further up the stack.

Access the encapsulated pointer by using the familiar pointer operators, `->` and `*`, which the smart pointer class overloads to return the encapsulated raw pointer.

The C++ smart pointer idiom resembles object creation in languages such as C#: you create the object and then let the system take care of deleting it at the correct time. The difference is that no separate garbage collector runs in the background; memory is managed through the standard C++ scoping rules so that the runtime environment is faster and more efficient.

Important

Always create smart pointers on a separate line of code, never in a parameter list, so that a subtle resource leak won't occur due to certain parameter list allocation rules.

The following example shows how a `unique_ptr` smart pointer type from the C++ Standard Library could be used to encapsulate a pointer to a large object.

C++

```
class LargeObject
{
public:
    void DoSomething(){}
};

void ProcessLargeObject(const LargeObject& lo){}
void SmartPointerDemo()
{
    // Create the object and pass it to a smart pointer
    std::unique_ptr<LargeObject> pLarge(new LargeObject());

    //Call a method on the object
    pLarge->DoSomething();

    // Pass a reference to a method.
    ProcessLargeObject(*pLarge);

} //pLarge is deleted automatically when function block goes out of scope.
```

The example demonstrates the following essential steps for using smart pointers.

1. Declare the smart pointer as an automatic (local) variable. (Do not use the **new** or `malloc` expression on the smart pointer itself.)
2. In the type parameter, specify the pointed-to type of the encapsulated pointer.
3. Pass a raw pointer to a **new**-ed object in the smart pointer constructor. (Some utility functions or smart pointer constructors do this for you.)
4. Use the overloaded `->` and `*` operators to access the object.
5. Let the smart pointer delete the object.

Smart pointers are designed to be as efficient as possible both in terms of memory and performance. For example, the only data member in `unique_ptr` is the encapsulated pointer. This means that `unique_ptr` is exactly the same size as that pointer, either four bytes or eight bytes. Accessing the encapsulated pointer by using the smart pointer overloaded * and -> operators is not significantly slower than accessing the raw pointers directly.

Smart pointers have their own member functions, which are accessed by using "dot" notation. For example, some C++ Standard Library smart pointers have a reset member function that releases ownership of the pointer. This is useful when you want to free the memory owned by the smart pointer before the smart pointer goes out of scope, as shown in the following example.

C++
```
void SmartPointerDemo2()
{
    // Create the object and pass it to a smart pointer
    std::unique_ptr<LargeObject> pLarge(new LargeObject());

    //Call a method on the object
    pLarge->DoSomething();

    // Free the memory before we exit function block.
    pLarge.reset();

    // Do some other work...

}
```

Smart pointers usually provide a way to access their raw pointer directly. C++ Standard Library smart pointers have a `get` member function for this purpose, and `CComPtr` has a public `p` class member. By providing direct access to the underlying pointer, you can use the smart pointer to manage memory in your own code and still pass the raw pointer to code that does not support smart pointers.

C++
```
void SmartPointerDemo4()
{
    // Create the object and pass it to a smart pointer
    std::unique_ptr<LargeObject> pLarge(new LargeObject());
```

```
    //Call a method on the object
    pLarge->DoSomething();

    // Pass raw pointer to a legacy API
    LegacyLargeObjectFunction(pLarge.get());
}
```

Kinds of Smart Pointers

The following section summarizes the different kinds of smart pointers that are available in the Windows programming environment and describes when to use them.

C++ Standard Library Smart Pointers

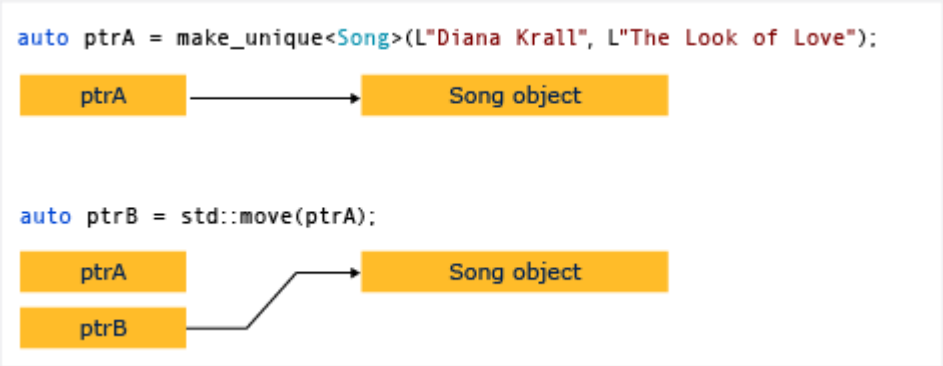
Use these smart pointers as a first choice for encapsulating pointers to plain old C++ objects (POCO).

- `unique_ptr`
  Allows exactly one owner of the underlying pointer. Use as the default choice for POCO unless you know for certain that you require a `shared_ptr`. Can be moved to a new owner, but not copied or shared. Replaces `auto_ptr`, which is deprecated. Compare to `boost::scoped_ptr`. `unique_ptr` is small and efficient; the size is one pointer and it supports rvalue references for fast insertion and retrieval from C++ Standard Library collections. Header file: `<memory>`. For more information, see How to: Create and Use unique_ptr Instances and unique_ptr Class.
- `shared_ptr`
  Reference-counted smart pointer. Use when you want to assign one raw pointer to multiple owners, for example, when you return a copy of a pointer from a container but want to keep the original. The raw pointer is not deleted until all `shared_ptr` owners have gone out of scope or have otherwise given up ownership. The size is two pointers; one for the object and one for the shared control block that contains the reference count. Header file: `<memory>`. For more information, see How to: Create and Use shared_ptr Instances and shared_ptr Class.
- `weak_ptr`
  Special-case smart pointer for use in conjunction with `shared_ptr`. A `weak_ptr` provides access to an object that is owned by one or more `shared_ptr` instances, but does not participate in reference counting. Use when you want to observe an object, but do not require it to remain alive. Required in some cases to break circular references between `shared_ptr` instances. Header file: `<memory>`. For more information, see How to: Create and Use weak_ptr Instances and weak_ptr Class.

How to: Create and Use unique_ptr Instances

A unique_ptr does not share its pointer. It cannot be copied to another `unique_ptr`, passed by value to a function, or used in any C++ Standard Library algorithm that requires copies to be made. A `unique_ptr` can only be moved. This means that the ownership of the memory resource is transferred to another `unique_ptr` and the original `unique_ptr` no longer owns it. We recommend that you restrict an object to one owner, because multiple ownership adds complexity to the program logic. Therefore, when you need a smart pointer for a plain C++ object, use `unique_ptr`, and when you construct a `unique_ptr`, use the make_unique helper function.

The following diagram illustrates the transfer of ownership between two `unique_ptr` instances.



`unique_ptr` is defined in the `<memory>` header in the C++ Standard Library. It is exactly as efficient as a raw pointer and can be used in C++ Standard Library containers. The addition of `unique_ptr` instances to C++ Standard Library containers is efficient because the move constructor of the `unique_ptr` eliminates the need for a copy operation.

Example

The following example shows how to create `unique_ptr` instances and pass them between functions.

C++

```
unique_ptr<Song> SongFactory(const std::wstring& artist, const std::wstring&
title)
{
    // Implicit move operation into the variable that stores the result.
    return make_unique<Song>(artist, title);
}

void MakeSongs()
{
    // Create a new unique_ptr with a new object.
    auto song = make_unique<Song>(L"Mr. Children", L"Namonaki Uta");

    // Use the unique_ptr.
```

```
    vector<wstring> titles = { song->title };

    // Move raw pointer from one unique_ptr to another.
    unique_ptr<Song> song2 = std::move(song);

    // Obtain unique_ptr from function that returns by value.
    auto song3 = SongFactory(L"Michael Jackson", L"Beat It");
}
```

These examples demonstrate this basic characteristic of `unique_ptr`: it can be moved, but not copied. "Moving" transfers ownership to a new `unique_ptr` and resets the old `unique_ptr`.

Example

The following example shows how to create `unique_ptr` instances and use them in a vector.

C++

```
void SongVector()
{
    vector<unique_ptr<Song>> songs;

    // Create a few new unique_ptr<Song> instances
    // and add them to vector using implicit move semantics.
    songs.push_back(make_unique<Song>(L"B'z", L"Juice"));
    songs.push_back(make_unique<Song>(L"Namie Amuro", L"Funky Town"));
    songs.push_back(make_unique<Song>(L"Kome Kome Club", L"Kimi ga Iru Dake
de"));
    songs.push_back(make_unique<Song>(L"Ayumi Hamasaki", L"Poker Face"));

    // Pass by const reference when possible to avoid copying.
    for (const auto& song : songs)
    {
        wcout << L"Artist: " << song->artist << L"   Title: " << song->title
<< endl;
    }
}
```

In the range for loop, notice that the `unique_ptr` is passed by reference. If you try to pass by value here, the compiler will throw an error because the `unique_ptr` copy constructor is deleted.

Example

The following example shows how to initialize a `unique_ptr` that is a class member.

C++

```
class MyClass
{
private:
    // MyClass owns the unique_ptr.
    unique_ptr<ClassFactory> factory;
```

```
public:

    // Initialize by using make_unique with ClassFactory default constructor.
    MyClass() : factory (make_unique<ClassFactory>())
    {
    }

    void MakeClass()
    {
        factory->DoSomething();
    }
};
```

Example

You can use make_unique to create a `unique_ptr` to an array, but you cannot use `make_unique` to initialize the array elements.

C++

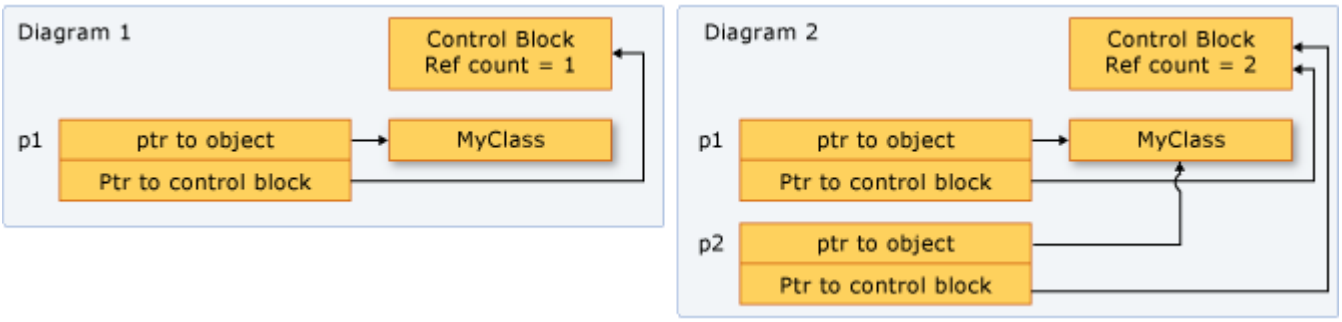
```
// Create a unique_ptr to an array of 5 integers.
auto p = make_unique<int[]>(5);

// Initialize the array.
for (int i = 0; i < 5; ++i)
{
    p[i] = i;
    wcout << p[i] << endl;
}
```

How to: Create and Use shared_ptr Instances

The `shared_ptr` type is a smart pointer in the C++ standard library that is designed for scenarios in which more than one owner might have to manage the lifetime of the object in memory. After you initialize a `shared_ptr` you can copy it, pass it by value in function arguments, and assign it to other `shared_ptr` instances. All the instances point to the same object, and share access to one "control block" that increments and decrements the reference count whenever a new `shared_ptr` is added, goes out of scope, or is reset. When the reference count reaches zero, the control block deletes the memory resource and itself.

The following illustration shows several `shared_ptr` instances that point to one memory location.



Example setup

The examples that follow all assume that you've included the required headers and declared the required types, as shown here:

C++

```
// shared_ptr-examples.cpp
// The following examples assume these declarations:
#include <algorithm>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct MediaAsset
{
    virtual ~MediaAsset() = default; // make it polymorphic
};

struct Song : public MediaAsset
{
    std::wstring artist;
    std::wstring title;
    Song(const std::wstring& artist_, const std::wstring& title_) :
        artist{ artist_ }, title{ title_ } {}
};

struct Photo : public MediaAsset
{
    std::wstring date;
    std::wstring location;
    std::wstring subject;
    Photo(
        const std::wstring& date_,
        const std::wstring& location_,
        const std::wstring& subject_) :
        date{ date_ }, location{ location_ }, subject{ subject_ } {}
};

using namespace std;

int main()
{
```

```
    // The examples go here, in order:
    // Example 1
    // Example 2
    // Example 3
    // Example 4
    // Example 6
}
```

Example 1

Whenever possible, use the make_shared function to create a `shared_ptr` when the memory resource is created for the first time. `make_shared` is exception-safe. It uses the same call to allocate the memory for the control block and the resource, which reduces the construction overhead. If you don't use `make_shared`, then you have to use an explicit `new` expression to create the object before you pass it to the `shared_ptr` constructor. The following example shows various ways to declare and initialize a `shared_ptr` together with a new object.

C++

```
// Use make_shared function when possible.
auto sp1 = make_shared<Song>(L"The Beatles", L"Im Happy Just to Dance With
You");

// Ok, but slightly less efficient.
// Note: Using new expression as constructor argument
// creates no named variable for other code to access.
shared_ptr<Song> sp2(new Song(L"Lady Gaga", L"Just Dance"));

// When initialization must be separate from declaration, e.g. class members,
// initialize with nullptr to make your programming intent explicit.
shared_ptr<Song> sp5(nullptr);
//Equivalent to: shared_ptr<Song> sp5;
//...
sp5 = make_shared<Song>(L"Elton John", L"I'm Still Standing");
```

Example 2

The following example shows how to declare and initialize `shared_ptr` instances that take on shared ownership of an object that has already been allocated by another `shared_ptr`. Assume that `sp2` is an initialized `shared_ptr`.

C++

```
//Initialize with copy constructor. Increments ref count.
auto sp3(sp2);

//Initialize via assignment. Increments ref count.
auto sp4 = sp2;

//Initialize with nullptr. sp7 is empty.
shared_ptr<Song> sp7(nullptr);
```

```
// Initialize with another shared_ptr. sp1 and sp2
// swap pointers as well as ref counts.
sp1.swap(sp2);
```

Example 3

`shared_ptr` is also helpful in C++ Standard Library containers when you're using algorithms that copy elements. You can wrap elements in a `shared_ptr`, and then copy it into other containers with the understanding that the underlying memory is valid as long as you need it, and no longer. The following example shows how to use the `replace_copy_if` algorithm on `shared_ptr` instances in a vector.

C++
```
vector<shared_ptr<Song>> v {
  make_shared<Song>(L"Bob Dylan", L"The Times They Are A Changing"),
  make_shared<Song>(L"Aretha Franklin", L"Bridge Over Troubled Water"),
  make_shared<Song>(L"Thalía", L"Entre El Mar y Una Estrella")
};

vector<shared_ptr<Song>> v2;
remove_copy_if(v.begin(), v.end(), back_inserter(v2), [] (shared_ptr<Song> s)
{
    return s->artist.compare(L"Bob Dylan") == 0;
});

for (const auto& s : v2)
{
    wcout << s->artist << L":" << s->title << endl;
}
```

Example 4

You can use `dynamic_pointer_cast`, `static_pointer_cast`, and `const_pointer_cast` to cast a `shared_ptr`. These functions resemble the `dynamic_cast`, `static_cast`, and `const_cast` operators. The following example shows how to test the derived type of each element in a vector of `shared_ptr` of base classes, and then copy the elements and display information about them.

C++
```
vector<shared_ptr<MediaAsset>> assets {
  make_shared<Song>(L"Himesh Reshammiya", L"Tera Surroor"),
  make_shared<Song>(L"Penaz Masani", L"Tu Dil De De"),
  make_shared<Photo>(L"2011-04-06", L"Redmond, WA", L"Soccer field at
Microsoft.")
};

vector<shared_ptr<MediaAsset>> photos;

copy_if(assets.begin(), assets.end(), back_inserter(photos), []
(shared_ptr<MediaAsset> p) -> bool
{
    // Use dynamic_pointer_cast to test whether
```

```
    // element is a shared_ptr<Photo>.
    shared_ptr<Photo> temp = dynamic_pointer_cast<Photo>(p);
    return temp.get() != nullptr;
});

for (const auto&  p : photos)
{
    // We know that the photos vector contains only
    // shared_ptr<Photo> objects, so use static_cast.
    wcout << "Photo location: " << (static_pointer_cast<Photo>(p))->location
<< endl;
}
```

Example 5

You can pass a `shared_ptr` to another function in the following ways:

- Pass the `shared_ptr` by value. This invokes the copy constructor, increments the reference count, and makes the callee an owner. There's a small amount of overhead in this operation, which may be significant depending on how many `shared_ptr` objects you're passing. Use this option when the implied or explicit code contract between the caller and callee requires that the callee be an owner.
- Pass the `shared_ptr` by reference or const reference. In this case, the reference count isn't incremented, and the callee can access the pointer as long as the caller doesn't go out of scope. Or, the callee can decide to create a `shared_ptr` based on the reference, and become a shared owner. Use this option when the caller has no knowledge of the callee, or when you must pass a `shared_ptr` and want to avoid the copy operation for performance reasons.
- Pass the underlying pointer or a reference to the underlying object. This enables the callee to use the object, but doesn't enable it to share ownership or extend the lifetime. If the callee creates a `shared_ptr` from the raw pointer, the new `shared_ptr` is independent from the original, and doesn't control the underlying resource. Use this option when the contract between the caller and callee clearly specifies that the caller retains ownership of the `shared_ptr` lifetime.
- When you're deciding how to pass a `shared_ptr`, determine whether the callee has to share ownership of the underlying resource. An "owner" is an object or function that can keep the underlying resource alive for as long as it needs it. If the caller has to guarantee that the callee can extend the life of the pointer beyond its (the function's) lifetime, use the first option. If you don't care whether the callee extends the lifetime, then pass by reference and let the callee copy it or not.
- If you have to give a helper function access to the underlying pointer, and you know that the helper function will just use the pointer and return before the calling function returns, then that function doesn't have to share ownership of the underlying pointer. It just has to access the pointer within the lifetime of the caller's `shared_ptr`. In this case, it's safe to pass the `shared_ptr` by reference, or pass the raw pointer or a reference to the underlying object. Passing this way provides a small performance benefit, and may also help you express your programming intent.

- Sometimes, for example in a `std:vector<shared_ptr<T>>`, you may have to pass each `shared_ptr` to a lambda expression body or named function object. If the lambda or function doesn't store the pointer, then pass the `shared_ptr` by reference to avoid invoking the copy constructor for each element.

Example 6

The following example shows how `shared_ptr` overloads various comparison operators to enable pointer comparisons on the memory that is owned by the `shared_ptr` instances.

C++

```
// Initialize two separate raw pointers.
// Note that they contain the same values.
auto song1 = new Song(L"Village People", L"YMCA");
auto song2 = new Song(L"Village People", L"YMCA");

// Create two unrelated shared_ptrs.
shared_ptr<Song> p1(song1);
shared_ptr<Song> p2(song2);

// Unrelated shared_ptrs are never equal.
wcout << "p1 < p2 = " << std::boolalpha << (p1 < p2) << endl;
wcout << "p1 == p2 = " << std::boolalpha <<(p1 == p2) << endl;

// Related shared_ptr instances are always equal.
shared_ptr<Song> p3(p2);
wcout << "p3 == p2 = " << std::boolalpha << (p3 == p2) << endl;
```

How to: Create and Use weak_ptr Instances

Sometimes an object must store a way to access the underlying object of a shared_ptr without causing the reference count to be incremented. Typically, this situation occurs when you have cyclic references between shared_ptr instances.

The best design is to avoid shared ownership of pointers whenever you can. However, if you must have shared ownership of shared_ptr instances, avoid cyclic references between them. When cyclic references are unavoidable, or even preferable for some reason, use weak_ptr to give one or more of the owners a weak reference to another shared_ptr. By using a weak_ptr, you can create a shared_ptr that joins to an existing set of related instances, but only if the underlying memory resource is still valid. A weak_ptr itself does not participate in the reference counting, and therefore, it cannot prevent the reference count from going to zero. However, you can use a weak_ptr to try to obtain a new copy of the shared_ptr with which it was initialized. If the memory has already been deleted, a bad_weak_ptr exception is thrown. If the memory is still valid, the new shared pointer increments the reference count and guarantees that the memory will be valid as long as the shared_ptr variable stays in scope.

Shared Pointers and Arrays

Before C++17, only `unique_ptr` was able to handle arrays out of the box (without the need to define a custom deleter). Now it's also possible with `shared_ptr`.

```
std::shared_ptr<int[]> ptr(new int[10]);
```

Please note that `std::make_shared` doesn't support arrays in C++17. But this will be fixed in C++20 (see P0674 which is already merged into C++20)

Another important remark is that raw arrays should be avoided. It's usually better to use standard containers. However, sometimes, you don't have the luxury to use vectors or lists - for example in an embedded environment, or when you work with third-party API. In that situation, you might end up with a raw pointer to an array. With C++17 you'll be able to wrap those pointers into smart pointers (`std::unique_ptr` or `std::shared_ptr`) and be sure the memory is deleted correctly.

Zestaw 9

1. Objaśnij działanie inteligentnych wskaźników: shared_ptr, unique_ptr

2. Sprawdź czy wielkość inteligentnego wskaźnika różni się od wielkości zwykłego wskaźnika.

3. Zaimplementuj szablon funkcji SongFactory tak aby obiekt ‘song’ był wskaźnikiem typu unique_ptr dla obiektów klasy Song

   ```
   auto song = SongFactory(L"Michael Jackson", L"Beat It");
   ```

   oraz przedstaw działanie konstruktora i destruktora obiektu przechowywanego przez wskaźnik we wnętrzu inteligentnego wskaźnika. Wypełnij std::vector<std::unique_ptr<Song>> songs i pokaż, że zakresowa pętla „for” musi wykorzystywać referencje.

4. Wyjaśnij zastosowanie dwóch możliwych specjalizacji dla szablonu unique_ptr

   template< class T, class... Args > unique_ptr<T> make_unique(Args&&... args);
   template< class T > unique_ptr<T> make_unique(std::size_t size);

5. Pokaż, że używanie wskaźnika typu shared_ptr zapewnia wywołanie destruktora w przypadku znikania ostatniego z nich.

6. Jakie jest działanie std::weak_ptr ? Przedstaw przykład użycia.